# SWEN30006 Software Modelling and Design
# Workshop 3: Design Models and Coding

School of Computing and Information Systems
University of Melbourne
Semester 1, 2025

**Assessment:** You *should* complete the exercises ***as a team***. To receive a workshop mark, you **need** to demonstrate your active participation during the workshop. Your tutor will observe your participation for you to be eligible to receive a mark for this workshop. The active participation include (but not limit to) present your solutions, help your teammates, and actively engage with the team. Attending the workshop with only passive participation will *not* be given a mark. See the Workshop Overview under Subject Information on the LMS for more details.

## Requisite Knowledge and Tools

To complete this workshop you will need to be familiar with the following terms and concepts:

- Use cases and use case diagrams
- Domain models and system sequence diagrams
- The representational gap
- Object Oriented Programming in Java

We assume that you have reviewed the **Pre-Workshop Learning** in Workshop 3 (Week 4) module in LMS when commencing this workshop activity.

## Workshop Objectives

This workshop will focus on the solution space and its implementation. This workshop aims to familiarise you with static and dynamic design modelling. We will be considering the relationship of the design models to the domain model and system sequence diagrams and the representational gap between the static models. We will also look at the translation of the design models into Java code. By the end of this workshop, you should:

- Be able to create static and dynamic design models based on use cases and domain models.
- Be able to use a UML notation for design class and design sequence diagrams
- Be able to implement a software system based on the design models.

**Note:** All tasks should be done as **a team** and demonstrate **active participation**.

# Part 1 From Domain to Design Class Diagrams

Having now generated a Domain Model for a subset of our new student learning management system, lets look at the work involved in deriving a Design Model from this Domain Model. Focusing on the `Upload an Assignment Submission` use case from the previous workshop, create a Design Model that specifies the software classes required for this functionality. You may abstract concepts such as Databases to a higher level. Assume the implementation is in Java.

After completing version 1 of your Static Design Model, answer the following questions:

1. When deriving our Design Model from the Domain Model (and other constraints) it is common that we try to minimise the *Representational Gap*. That is, we translate a domain class `Exam` to a design class `Exam`. We could have easily called the design class `BaseBallGlove` or `AhKisuLwjMMn` but we don't. What advantages does this give?
2. What kind of assumptions did you need to make about the design to construct the Design Model from the Domain Model? Where would you normally get this kind of data?

# Part 2 Creating a Dynamic Design Model

Starting from the `Submission` System Sequence Diagram, draw a `Submission` design sequence diagram. Ensure that your system sequence diagram, your design class diagram, and your design sequence diagram are consistent; you may need to iterate a few times making changes to achieve this.

After completing version 1 of your Dynamic Design Model, answer the following questions:

1. What does the dynamic model tell us about the static model?
2. Would it have been better to start with the dynamic model and use it to inform construction of the static model?

# Part 3 From Design to Implementation

The final step after you have finished your static modelling is to move to actual implementation. Often it is the case that developers will focus on Static models (including domain and design class diagrams), along with use cases, to specify how software is to be written. Figures 1 and 2 specify the domain and class diagrams for a basic program that validates student submissions to our new LMS system.

The required interface and a sample mock implementation has been provided in the Github assignment: https://classroom.github.com/a/p9ELNYOj. Note that this GitHub assignment has GitHub Actions for automated build (but no automated tests).

**Code Collaboration:** Everyone in a team work collaboratively. You should use this exercise to practice dividing up the coding work in an effective way.

**How to use GitHub assignment?** You can refer to instructions on how to access GitHub assignments and use Git in the Workshop 0 sheet.

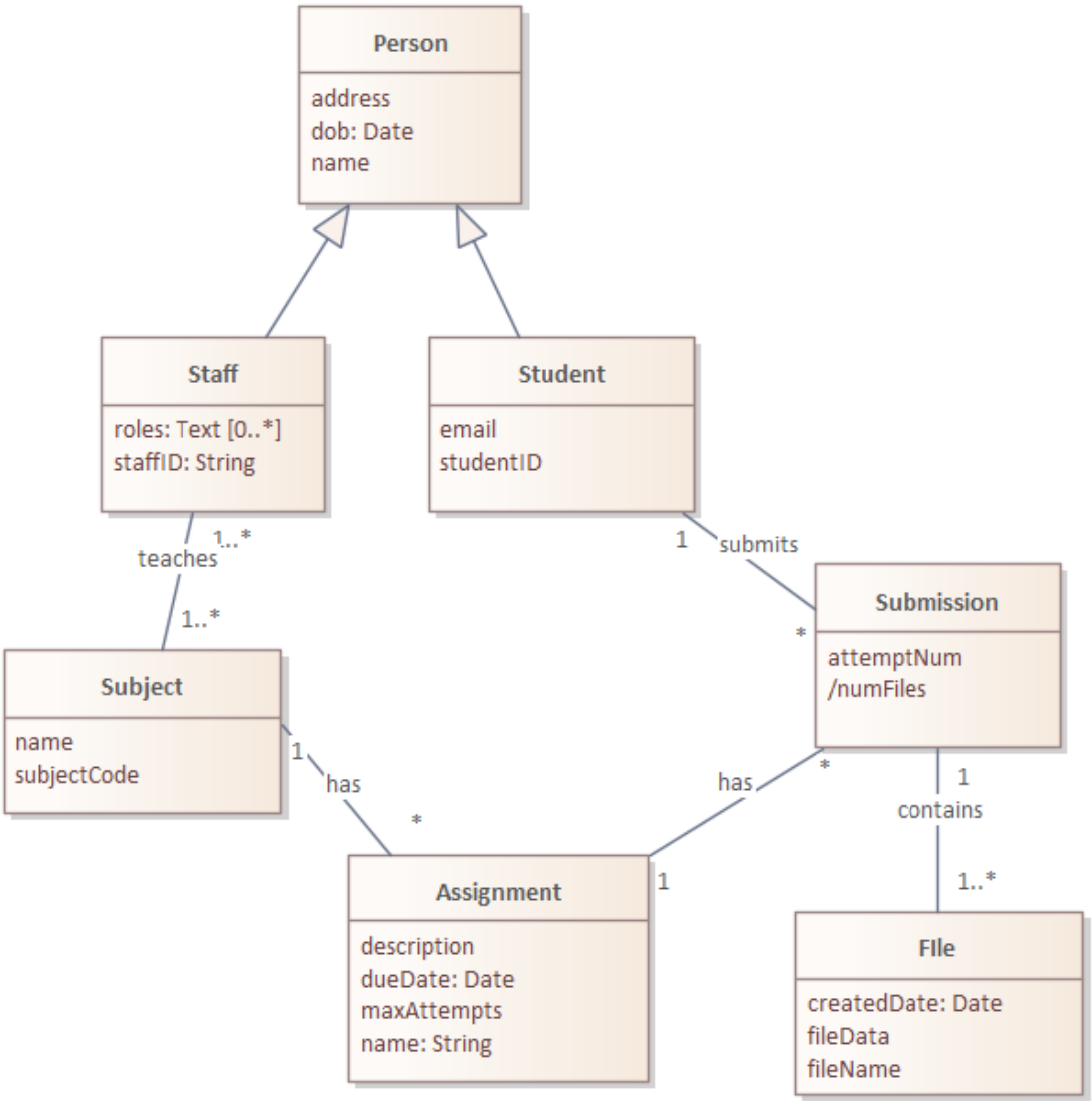


Figure 1: Incomplete Domain Model for Submission Validation

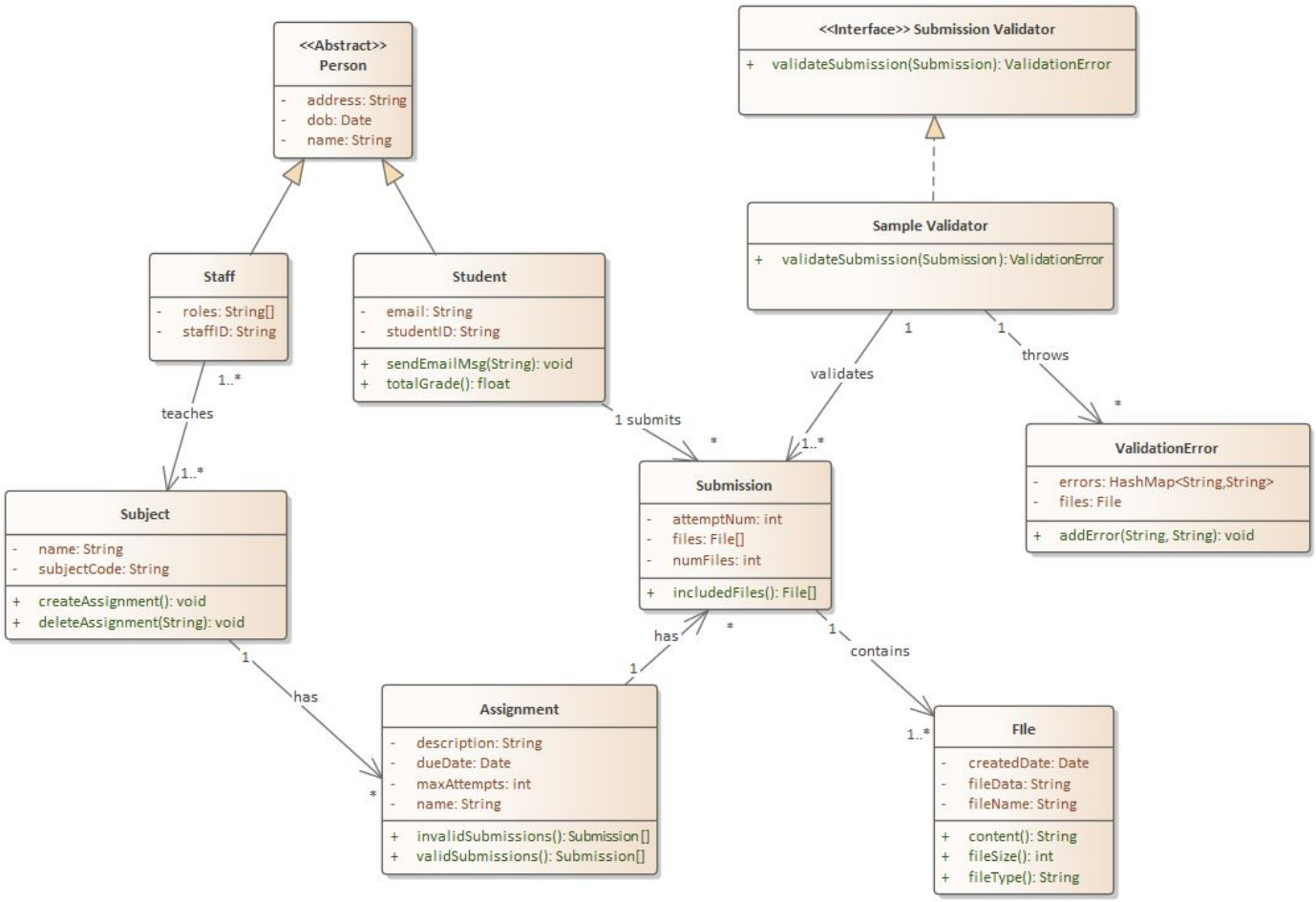


Figure 2: Incomplete Design Model for Submission Validation

Based on this information alone, create a Java implementation of the Design Model in Eclipse, along with a driver program to test your program. Once you have completed this, answer the following questions.

1. Did you have sufficient information to complete the implementation?
2. What assumptions, if any, did you have to make about certain functions or relationships within the program?
3. Does your program work the same as people's sitting near to you? If not, how does it differ?